

Kushagra Jaiswal

Machine Learning Engineer

Faridabad, Haryana, India | +91 8448328995 | kinuj768@gmail.com
linkedin.com/in/kushagra356 | github.com/Nerd-coderZero | nerd-coderzero.github.io

SUMMARY

ML Engineer building and deploying production ML systems — from research architectures (CNN-Transformer hybrids, CBAM attention) to Kubernetes-served pipelines and evaluated RAG systems, with MLflow tracking, monitoring, and load-tested serving layers. Focused on shipping work that is verifiable: live deployments, public repositories, and documented evaluation.

EXPERIENCE

Independent ML Engineering

May 2026 – Present

Personal Projects — Faridabad, India

- Built and shipped four ML systems end to end, all publicly verifiable: a retail demand forecasting service, a RAG system with a frozen-test evaluation, a live manga translation pipeline, and an agentic evaluation harness.
- Closed production-serving gaps deliberately: Kubernetes deployment, MLflow tracking and model registry, FastAPI serving with request batching, Prometheus monitoring and load testing under concurrent traffic.

ML Engineer & Research Lead

May 2025 – May 2026

Megaminds IT Services — Remote

- Took research architectures to working implementations: CNN-Transformer hybrids with cross-attention fusion, CBAM spatial/channel attention on ResNet backbones, and federated learning pipelines.
- Reached 97% accuracy on histopathology classification and 89% on multi-class MRI classification.
- Directed technical delivery across a team of 10+ research associates: project assignment, technical mentorship, onboarding, and performance review.
- Handled client communication and requirements directly on delivered projects.
- Domains: medical imaging, NLP classification, RAG pipelines, GAN-based augmentation, AutoML, federated learning.

SELECTED PROJECTS

Retail Demand Forecasting — Kubernetes, MLflow, Production Serving

Personal Project

- Seven-day demand forecasting for 50,000 retail store-product series (FreshRetailNet-50K, CC BY 4.0). LightGBM model with leakage-tested features, enforced by 9 unit tests and a serving-parity check that reproduces all 350,000 evaluation predictions exactly.
- Scored once on a held-out evaluation week at 0.3317 WAPE: 8.2% better than the strongest classical baseline (0.3612) and 5.1% better than a zero-shot foundation model (Chronos-2, 0.3494).
- Served via FastAPI on Kubernetes (kind), with MLflow experiment tracking and model registry, Prometheus monitoring, and locust load testing.
- Request batching measured at 3.0x throughput and a 4x lower p99 at 50 concurrent users; the service sustains 82 rps at p50 100 ms on a single CPU pod at 10 users. Both verified in-cluster against the running deployment and the service's own Prometheus metrics.

Stack: Python, LightGBM, MLflow, FastAPI, Docker, Kubernetes, Prometheus, locust

Code: github.com/Nerd-coderZero/retail-demand-forecasting

RAG over Apache Airflow Documentation — Retrieval Evaluation, Local LLM

Personal Project

- Retrieval-augmented question answering over a pinned Airflow documentation snapshot (1,173 files, 570,658 words), run fully locally: BGE embeddings with FAISS dense retrieval, Qwen2.5-7B via Ollama, served through FastAPI.
- Evaluation designed before the pipeline: 55 questions (direct, multi-source and unanswerable) verified against the pinned corpus, split 15 dev / 40 frozen test; every design choice made on dev, test set scored once.
- Retrieval lifted answer accuracy from 47% to 62% on the frozen test set (same model and questions, no-retrieval baseline), and raised correct refusals on unanswerable questions from 0 of 7 to 4 of 7.
- Configuration chosen by dev-set measurement: fixed-window chunking beat structure-aware chunking, and dense retrieval outranked BM25 and hybrid (RRF); a context-window check caught Ollama silently truncating long prompts to about 2,050 tokens before the RAG runs.

Stack: Python, FAISS, BGE embeddings, BM25, Ollama (Qwen2.5-7B), FastAPI, MLflow, pytest

Code: github.com/Nerd-coderZero/airflow-docs-rag

Manga & Comic Translation Pipeline

Personal Project

- End-to-end Chinese-to-English and Japanese-to-English manga translation pipeline: text detection, OCR, LLM-based translation, and in-image text placement.
- Hybrid detection/recognition approach for Japanese — PaddleOCR for detection, manga-ocr (purpose-built model) for recognition — because general-purpose OCR handles vertical manga text poorly.
- Three-tier text placement logic that fits translated text at original size, expands into available space when needed, or skips placement rather than force an illegible result.
- Deployed on Hugging Face Spaces (Gradio SDK, ZeroGPU); backend built on a FastAPI service layer reused across both the deployed app and an async batch API.
- 155 automated checks (49 pipeline_core, 44 API, 44 browser, 18 Gradio) run against a stubbed pipeline, with live end-to-end Japanese translation confirmed against the deployed Space.

Stack: Python, PaddleOCR, manga-ocr, NVIDIA NIM (Nemotron), FastAPI, Gradio, Hugging Face Spaces (ZeroGPU)

Live: huggingface.co/spaces/NerdCoderZero/manga-translator | **Code:** github.com/Nerd-coderZero/Manga-Translator

Agentic Evaluation Harness for LLM-in-the-Loop Pipelines

Personal Project

- LangGraph-based evaluation harness that stress-tests the Manga Translation Pipeline's live deployment: runs batches of real inputs, scores outputs against defined quality checks, and decides to retry, fall back, or flag for review — logging every decision with its reasoning.
- Found three real, previously undocumented failure modes on the live system: a garbage-OCR passthrough gap in the upstream filter, an LLM meta-commentary leak, and a content-policy refusal case requiring the pipeline to distinguish a correct model refusal from a translation failure.
- All three findings locked in as regression fixtures, with a documented design rationale mirroring the source pipeline's own decision-log format.

Stack: Python, LangGraph, Gradio Client

Code: github.com/Nerd-coderZero/Manga-Translator-Eval-Harness

eBPF + ML Runtime Threat Detection for Kubernetes

Client Project

- Runtime security system for Kubernetes nodes: an eBPF/BCC node agent streaming syscall and network events to an ML detection service that scores container behaviour for anomalies.
- Detection model is an LSTM autoencoder combined with a supervised ensemble, trained and evaluated on a simulated corpus of benign and attack traces spanning five attack classes.
- Training pipeline instrumented with MLflow experiment tracking and a model registry; deployed to a Kubernetes cluster with RBAC, configmap and deployment manifests, verified end to end from image build through pod health to live inference on held-out traces.

Stack: Python, eBPF/BCC, PyTorch, scikit-learn, MLflow, Docker, Kubernetes

TECHNICAL SKILLS

Frameworks & Libraries	Python, PyTorch, Scikit-learn, LightGBM, LangGraph, FAISS
Architectures	CNN-Transformer hybrids, CBAM attention, LSTM autoencoders, Federated Learning, GANs, AutoML
MLOps & Serving	MLflow (tracking & model registry), FastAPI, Docker, Kubernetes, Prometheus, locust, Ollama
Domains	Medical Imaging, NLP & LLMs, Retrieval-Augmented Generation, Time-Series Forecasting
Evaluation	ROC/AUC, Macro/Micro F1, WAPE, hit@k/MRR, closed-book baselines, frozen test sets, ablation studies, leakage-parity testing, temporal train/test splits
Tools & Infrastructure	Git, REST API, Jupyter, Hugging Face Spaces

EDUCATION

BTech, Computer Science Engineering

2021

J.C. Bose University of Science and Technology, YMCA

Post Graduate Program, Data Science Bootcamp

2023

UpGrad / Golden Gate University, San Francisco